

Course Code: **IDC 222**



Scientific Computing & Data Visualization

Reji J

School of Data Science

IISER Thiruvananthapuram

1. C++ Character Set

- Character set is a set of valid characters that are recognized by the language.
- The Character set of C++ is categorized as follows:
 - Letters: **A - Z, a-z**
 - Digits: **0 - 9**
 - Special Characters: **~ ! @ # \$ ^ & () { } [] _ | \ : ; ' " , . ? + - * / = < >**
 - White Spaces: Blank space, Horizontal Tab, Carriage Return
 - Other Characters: Non graphic symbols and ASCII characters.



2. Tokens

- Tokens are the smallest individual units in a program.

- C++ has **five** tokens:

→ Keywords

→ Identifiers

→ Literals

→ Punctuators

→ Operators



2.1 Keywords

- Keywords are the words (tokens) that convey specific meaning to the compiler.
- aka **reserved words** because they are reserved by the language for special purposes and cannot be redefined for any other purpose.

Table 9.1 C++ Keywords

asm	auto	break	case	catch
char	class	const	continue	default
delete	do	double	else	enum
extern	float	for	friend	goto
if	inline	int	long	new
operator	private	protected	public	register
return	short	signed	sizeof	static
struct	switch	template	this	throw
try	typedef	union	unsigned	virtual
void	volatile	while		



2.2 Identifiers

- Identifiers are the words that identify different program elements such as memory locations (**variables**), **functions** (function names), **statements** (label) etc.
- Each language has its own rules for naming these identifiers.
- The following rules are common:
 - Identifiers is an arbitrary long sequence of letters and digits.
 - The first character must be a letter.
 - The underscore (_) counts as a letter.
 - White spaces are not allowed.
 - Upper case and lower case letters are different.
 - Keywords cannot be used as identifiers.



Contd.....

- The following are some of valid identifiers

Num **_avg_hgt** **Program**

var **addr2** **Int**

realnum **FILE**

- The following are invalid identifiers:

Stud-rec The hyphen character - is used.

2number Digit is used as first character

int Keyword (already reserved by the language)



2.3 Literals

- Literals are the tokens that represent data items that never change their value during a program run.
- They are often referred to as **constants**.
- C++ has four types of literals:
 - Integer Literals
 - Character Literals
 - Floating Literals
 - String Literals



Contd...

- **Integer Literals**

- Integers Constants are the whole numbers constituted by the digits.
- it must have at least one digit and must not have any decimal point
- may contain + or - symbol
- Decimal (base 10) - a sequence of digits with no zeros in the beginning
- Octal (Base 8) - A sequence of digits starting with 0 (zero).
- Eg: 023, 045
- Hexadecimal (Base 16) - a sequence of digits preceded by 0x or 0X.

Eg: 0X79, 0XAB



Contd...

- **Character Literals**

- a valid character enclosed in double quotes.
- C++ support certain non-graphic characters
- backspace, tabs, carriage return - represented by using escape sequences which consists of a backslash (\) followed by one or more characters.

Table 9.2 Escape Sequences

Escape sequence	Non-graphical character
\a	Audible or alert bell
\b	Backspace
\f	Form feed
\n	Newline or linefeed
\r	Carriage return
\t	Horizontal tab
\v	Vertical tab
\\	Backslash
\'	Single quote
\"	Double quote
\?	Question Mark
\On	Octal number
\xHn	Hexadecimal number
\0	Null



- **Floating Constants**

- Floating constants are the real numbers (numbers with fractional part).

- They can be written in two forms:

- Fractional Form**

- Exponential Form**

- A floating constant in **Fractional form** must have at least one digit before the decimal point (period symbol) and at least one digit after the decimal point.

- It have a + or - sign preceding it.

- A real constant without any sign is treated as positive.

- The following are valid floating constants in fractional form:

4.5 123.0 0.78 -0.0025 +21.0

- The following are invalid floating constants in fractional form:

21 No decimal point

2.5.8

Two decimal points

3. No digit after decimal point

2,345.23

Comms not allowed

+.01 No digit before decimal point



- **Floating Constants**

- A floating constant in **Exponent form** has two parts.

- **a mantissa**, which must be an integer.

- or a **proper real constant** and **exponent**, which must be an integer.

- The mantissa is followed by the letter **e** or **E** and the exponent.

4.5 123.0 0.78 -0.0025 +21.0

- The following are valid real constants in exponential form:

12e-5 (equivalent to **12 x 10⁻⁵**), **12.7E10** (**12.7 x 10¹⁰**), **0.34e09**, **-6.7E-9** etc.

- The following are invalid real constants in exponential form:

45.e4 Mantissa has no digit after decimal point

4.5E No exponent

2.3e4.5 Decimal point is not allowed in exponent



Contd...

- **String Literals**

- String Constants is a sequence of characters enclosed in double quotes.
- They are the **multi-character constants**.
- But multi character constants such as octal number and hexadecimal number are different as they are assigned with ASCII numbers, which are integers
- When a string literal is stored in memory, it will be delimited by a null character (**\0**), called the **string terminator**.
- size of a string is the number of characters contained in it **plus one**.
- **Eg:** The size of “C++” is **4** and “hello guys” is **11**.



2.4 Punctuators

- Punctuators are the symbols used as the punctuation marks in the syntax of various constructs and a separators to enhance the program's readability.
- The following characters are used as punctuators in C++:
- **Brackets [] Parentheses () Braces { }**
(These punctuators are used in pairs - one as opening and other as closing)

- Comma ,
- Ellipse ...
- Semicolon ;
- Equal to =
- Colon :
- Pound sign or Hash symbol #
- Asterisk *



2.5 Operators

- Operators are tokens or symbols that trigger some kind of operation such as input, output, computation, assignment etc.
- Operators are mainly classified into,
 - Arithmetic operators
 - Relational operators
 - Logical operators
 - special operators such as input, output, assignment.
- Eg:

+ - * / % << >> ++ etc



3. Operators

- The tokens (or symbols) that trigger some kind of operations are called *operators*.
- An operation requires an *operator* and *operand*.
- *Operands* are the data on which the specified task is carried out.
- *Operators* are used in programs to manipulate data and variables,
- C++ operators can be classified into a number of categories. However, there is a general classification *based on the number of operands required for an operation*.
- They are **unary**, **binary** and **ternary**.
- **Unary** operator requires one operand to operate upon.
- **binary** requires two operands.
- **ternary** operator requires three operands to operate upon.
- But the usual classification of operators is based on the *type of operation performed*. The detailed classification is given in the following sections:



3.1 I/O Operators

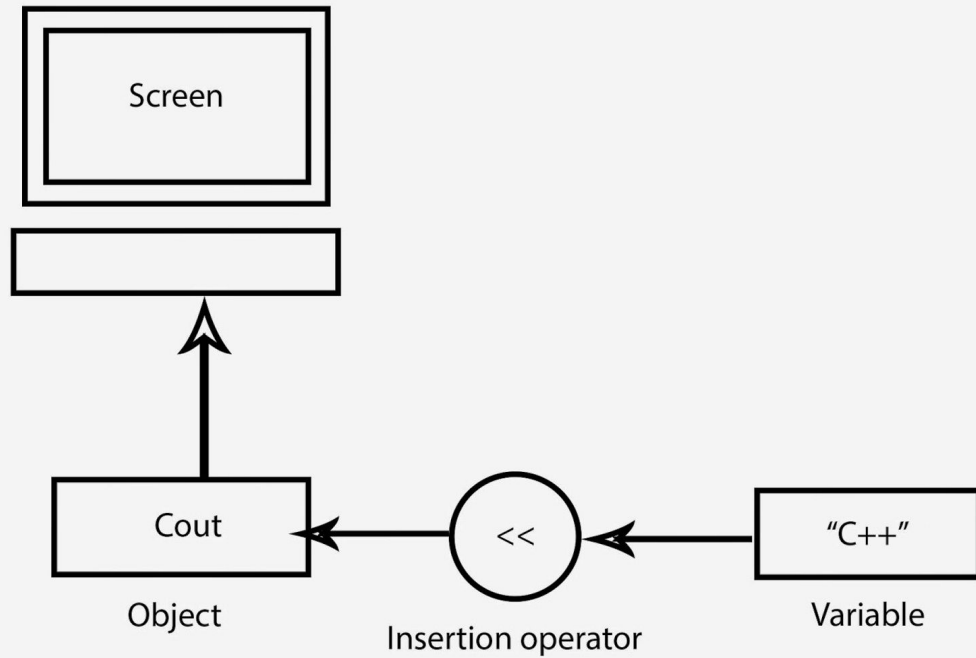
- The standard input and output devices (keyboard and monitor) are tied to a predefined stream in C++ and they are identified as the objects **cin** and **cout** respectively.
- The operators for input and output operations are,
- **>>** (called **extraction operator** or **get from operator**)
- **<<** (called **insertion operator** or **put to operator**)
- When input operation is performed, data is encountered through the keyboard and it enters the memory. Hence the variables will be used to refer these data.

cin >> num1;

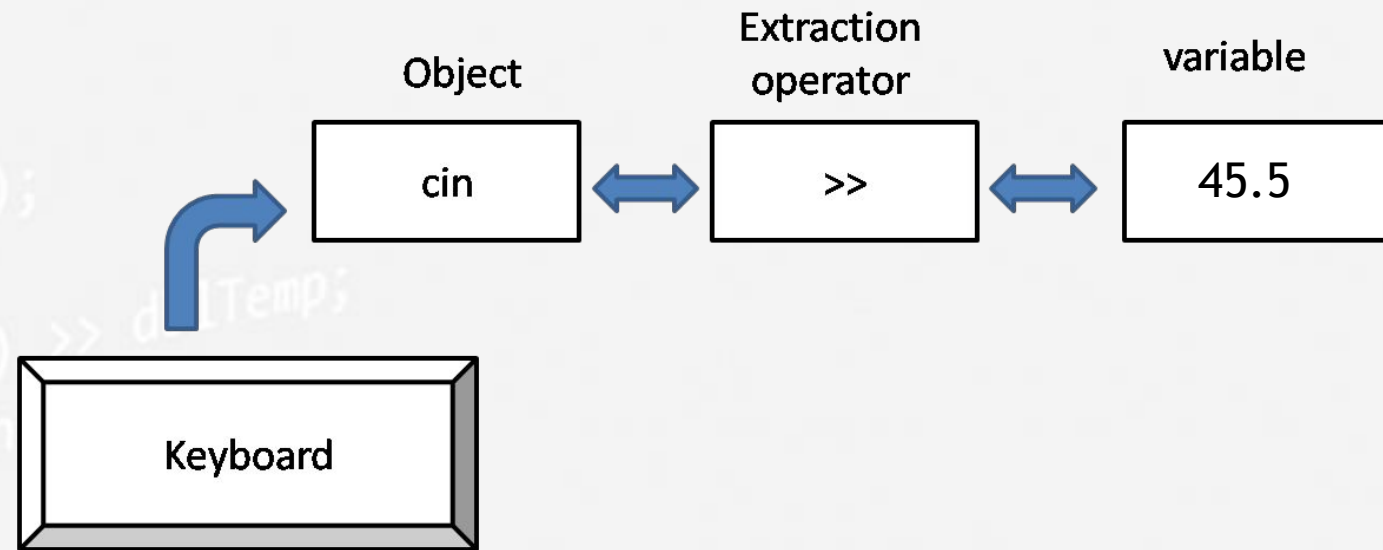
- This is an input statement and it causes the program to wait for the user to type in a number.
- The number keyed in is placed in the variable num1.
- The identifier **cin** is a predefined object in C++ that corresponds to the standard input stream, that is keyboard.
- The name **cin** stands for '**console input**'. The operator **>>** is known as **extraction operator**.
- it extracts (or takes) the value from the keyboard assigns it to the variable on its right)



Use of I/O Operators



Output using insertion operator



Input using extraction operator



Cascading of I/O Operators

- Consider the statements

```
cin>>num1;
```

```
cin>>num2;
```

- The two statements are used to input two numbers and store them each in the variables num1 and num2.
- The same can be achieved by the following statement:

```
cin>>num1>>num2;
```

- Similarly output operations can be performed as,

```
cout<<"sum of"<<num1<<"and"<<num2<<"is"<<sum;
```

Note that a single statement contains more than one input & output operator and variables.

- The multiple use of >> (**get from**) operator in one statement is called *cascading of input operators*
- The multiple use of << (**put to**) operator in one statement is called *cascading of output operators*



3.2 Arithmetic Operators

- The symbols (or tokens) that trigger computations in the program are called *arithmetic operators*.
- There are **unary** and **binary** *arithmetic operators*
- **unary** + and **unary** - are the unary operators and they make numbers positive or negative.
- The **binary** *arithmetic operators* are: + - * / %
- Except the % (**modulus operator**), all others work with both integer and floating point data.
- The % operator works only on integer operands.

Operators	Meaning	Example	Result
+	Addition	4+2	6
-	Subtraction	4-2	2
*	Multiplication	4*2	8
/	Division	4/2	2
%	Modulus operator to get remainder in integer division	5%2	1



Increment / Decrement Operators

- C++ provides two special arithmetic operators, namely increment operators(++) and decrement operators(- -) .
- Both these are unary operators and the operand should be a numeric (int, float or char) variable.
- The **increment operator (++)** **adds 1** to its operand and the **decrement operator (--)** **subtracts 1** from its operand.
- The result will be stored in the same operand and that is why it should be a variable.
- Consider the code snippet:

```
int a = 5, b = 3;
```

```
a++;
```

```
b--;
```

- The values of a and b will be **6** and **2** respectively after the execution of these statements.
- i.e.; **a++;** is equivalent to **a = a+1;** and **b--;** is same as **b = b-1;**



Contd...

- There are two versions of these operators: **postfix** and **prefix**.
- In **postfix notation**, the operator is placed after the operand as in **a++** and **b--**;
- but in **prefix notation**, the operator precedes the operand as in **++a** and **--b**.
- Both are **same** as far as the variables **a** and **b** are concerned. But these notations differ when these are used with assignment statements.

For example,

- **b = a++;** and **b = ++a;** are **different**
- In the first case **a = 6** and **b = 5**, but in the latter case both **a** and **b** will be **6**.
- The statement **b = a++;** is equivalent to the statement sequence
b = a;
a = a+1;
- The current value of **a** is used (**for assignment**) **first** and **then it is changed to a +1**
(hence known as **use and change method**)



Contd...

- But the second statement **b = ++a;**
- is equivalent to the statement sequence **a = a+1; b=a;**
- Here, the value of **a** is **changed first** and the **new value is used for assigning** (hence known as **change and use method**).
- Similar is the case for **decrement operator...**



3.3 Relational Operators

- The operators that are compare two quantities are called *relational operators*.
- They determine the relation between the two data.
- The result of the operation will be either **TRUE (1)** or **FALSE (0)**.
- There are six such operators and all of them are binary operators.

OPERATOR	MEANING	EXAMPLE	RESULT
<	Less than	1<2	True
>	Greater than	1>2	False
<=	Less than or equal to	1<=2	True
>=	Greater than or equal to	1>=2	False
==	Equal to	1==2	False
!=	Not equal to	1!=2	True



3.4 Logical Operators

- Logical operators are used with expressions. The operands as well as the result of these operators are Boolean values (**TRUE** and/or **FALSE**).
- The **logical AND** (**&&**), **logical OR** (**||**) and **logical NOT** (**!**) are the operators.
- When we want to test more than one condition and make decisions, **&&** and **||** operators are used and hence they are **binary operators**.
- But **logical NOT** (**!**) is a **unary operator** as works with **one relational operation**.
- Eg:
 (a < b) && (b < c) uses **&&** operator to **combine two relational operations** (expressions) and hence a **logical operation** (expression) is referred to as a ***compound relational operation (expression)***.
- Since the relational operations results in to **TRUE** or **FALSE** values, the operands of the logical expressions will be the same.



OPERATOR	MEANING	EXAMPLE	RESULT
&&	Logical AND	(1<2) && (2>1)	True
	Logical OR	(1<2) (2>1)	True
!	Logical NOT	!(1<2)	False

&&

Operand 1	Operand 2	Result
True	True	True
True	False	False
False	True	False
False	False	False



||

Operand 1	Operand 2	Result
True	True	True
True	False	True
False	True	True
False	False	False

!

Operand	Result
False	True
True	False



3.5 Assignment Operator

- The assignment operator in C++ assigns a value provided at the right hand side of the is used as the assignment operator to the variable provided at the left hand side and symbol = is used as the assignment operator.
- It is also a binary operator, as it needs a value as one operand and a variable as the second operand.
- The operand to the left of = should be a variable. By assigning a value to a variable, we actually mean the value is stored in the location identified by the variable.



3.6 Conditional Operator

- *The conditional operator returns a value depending on a condition.*
- It is a **ternary operator**, which requires three operands to operate upon. The syntax is:

expression 1 ? expression 2: expression 3

- The **expression1** represents a condition and if it evaluates to TRUE, then the result of the whole **expression (operation)** will be the value returned by the **expression2** (i.e.; the output of the operation specified in expression 2); otherwise, the result will be the value returned by the **expression3**.
- Consider the C++ statement:
grade = (avg>60) ? 1 : 0;
- If the content of **avg** is greater than **60** then **1** will be stored in the variable *grade* otherwise **0 (zero)** will be stored in *grade*.
- Similarly the following statement uses conditional operator.

val = (a>b) ? a : a-b;



3.7 sizeof Operator

- *Sizeof is a unary compile-time operator that returns the number of bytes occupied by the operand.*
- The operand may be a variable, a constant or data type and it is specified within the parentheses.
- For example, the result of the operation **sizeof(float)** is **4** as float data type take **4 bytes of memory**.

3.8 Comma Operator

- The comma operator is a special type of operator and is used to string together several expressions.
- The group of expressions separated by commas will be evaluated from left to right in sequence and the result of the right most expression will be the value of the total comma separated expression.
- Hence it is also known as separator. The statement

b = (a = 1, a + 1); will assign 1 to a first and then evaluates a+1 to 2

which will be stored in b

